

Taller en Google Colab: Chunking para bases de datos vectoriales con Neon + pgvector

Este cuaderno está diseñado para que los estudiantes **implementen, comparen y evalúen** varias estrategias de *chunking* sobre un corpus de ejemplo y luego almacenen los embeddings en **Neon (PostgreSQL serverless) con pgvector**.

Propósito pedagógico

Al final del taller, cada estudiante debería poder responder con evidencia:

1. **Cuál estrategia produce chunks más coherentes.**
2. **Cuál estrategia mejora el retrieval** para cierto tipo de consulta.
3. **Cuándo conviene usar:**
 - `fixed-size`
 - `sentence-aware`
 - `semantic`
4. Cuál es el costo computacional que implica cada alternativa.

Qué haremos

- Crear y poblar una tabla en Neon.
- Aplicar tres estrategias de chunking.
- Generar embeddings con `all-MiniLM-L6-v2`.
- Insertar chunks y vectores en Neon.
- Ejecutar búsquedas semánticas.
- Comparar estrategias con métricas simples de retrieval.

0. Antes de empezar

Requisitos

- Tener una cuenta gratuita en **Neon**.
- Crear un proyecto nuevo.
- Copiar el `DATABASE_URL` de conexión.
- Haber habilitado la extensión `vector` en Neon o permitir que este notebook lo haga por SQL.

Recomendación didáctica

Los estudiantes pueden trabajar en parejas y **no cambien todas las variables al tiempo**.

Primero comparen **una sola estrategia**, luego ajusten tamaño, overlap o threshold.

```
1 # Instalación de dependencias
2 !pip -q install psycopg2-binary pgvector sentence-transformers nltk spacy scikit-learn pandas matplotlib lan
3 !python -m nltk.downloader punkt punkt_tab >/dev/null 2>&1
4 !python -m spacy download es_core_news_sm >/dev/null 2>&1
```

4.2/4.2 MB 40.8 MB/s eta 0:00:00

```
1 import os
2 import json
3 import getpass
4 import warnings
5 warnings.filterwarnings("ignore")
6
7 import numpy as np
8 import pandas as pd
9 import matplotlib.pyplot as plt
10
11 import nltk
12 import spacy
13 from nltk.tokenize import sent_tokenize
14 from sklearn.metrics.pairwise import cosine_similarity
15
16 import psycopg2
17 import psycopg2.extras
18 from pgvector.psycopg2 import register_vector
19
20 from sentence_transformers import SentenceTransformer
21 from langchain_text_splitters import RecursiveCharacterTextSplitter
22
23 pd.set_option("display.max_colwidth", 120)
```

✓ 1. Configuración de conexión a Neon

Pega aquí tu `DATABASE_URL`.

Debe verse parecido a esto:

```
postgresql://usuario:password@ep-xxxx.region.aws.neon.tech/neondb?sslmode=require
```

Neon documenta el uso de `pgvector` como extensión para almacenar embeddings y hacer búsquedas vectoriales en PostgreSQL. [cite](#) [turn455150search0](#) [turn455150search14](#)

```
1 import getpass
2 DATABASE_URL = getpass.getpass("Pega aquí tu DATABASE_URL de Neon: ").strip()
3 if not DATABASE_URL:
4     raise ValueError("Debes proporcionar DATABASE_URL para continuar.")
5 print("Connection string capturado correctamente.")
```

Pega aquí tu DATABASE_URL de Neon:
Connection string capturado correctamente.

```
1 import psycopg2
2 # 1) Conectar SIN register_vector
3 conn = psycopg2.connect(DATABASE_URL)
4 conn.autocommit = True
5 cur = conn.cursor()
6 # 2) Habilitar la extensión en esa base
7 cur.execute("CREATE EXTENSION IF NOT EXISTS vector;")
8 # 3) Verificar
9 cur.execute("SELECT extname FROM pg_extension WHERE extname = 'vector';")
10 print(cur.fetchone())
11
12 cur.close()
13 conn.close()
```

('vector',)

```
1 def get_conn():
2     conn = psycopg2.connect(DATABASE_URL)
3     register_vector(conn)
4     return conn
5
6 # Prueba rápida de conexión
7 conn = get_conn()
8 cur = conn.cursor()
```

```

9 cur.execute("SELECT version();")
10 print(cur.fetchone()[0][:120], "...")
11 cur.close()
12 conn.close()

```

PostgreSQL 17.8 (6108b59) on aarch64-unknown-linux-gnu, compiled by gcc (Debian 12.2.0-14+deb12u1) 12.2.0, 64-bit ...

✓ 2. Crear esquema en Neon

Usaremos una sola tabla para almacenar todos los chunks de todas las estrategias.

El modelo `all-MiniLM-L6-v2` produce embeddings de **384 dimensiones**. `□cite□turn455150search10□turn455150search7□`

```

1 SCHEMA_SQL = '''
2 CREATE EXTENSION IF NOT EXISTS vector;
3
4 CREATE TABLE IF NOT EXISTS chunks (
5     id                BIGSERIAL PRIMARY KEY,
6     doc_id            TEXT NOT NULL,
7     doc_type          TEXT NOT NULL,
8     strategy          TEXT NOT NULL,
9     chunk_index       INTEGER NOT NULL,
10    content            TEXT NOT NULL,
11    token_count        INTEGER,
12    metadata           JSONB,
13    embedding          vector(384),
14    created_at         TIMESTAMPTZ DEFAULT NOW()
15 );
16
17 CREATE INDEX IF NOT EXISTS chunks_doc_strategy_idx
18     ON chunks (doc_id, strategy);
19
20 CREATE INDEX IF NOT EXISTS chunks_hnsw_idx
21     ON chunks USING hnsw (embedding vector_cosine_ops);
22 '''
23
24 with get_conn() as conn:
25     with conn.cursor() as cur:
26         cur.execute(SCHEMA_SQL)
27     conn.commit()

```

28

Esquema creado o verificado correctamente.

3. Corpus base del taller

Para que el notebook funcione sin archivos externos, incluimos un pequeño corpus base con cuatro tipos de documento:

- DOC-1 : paper / texto académico
- DOC-2 : noticia
- DOC-3 : manual técnico
- DOC-4 : FAQ

Si lo desean pueden cambiar los documentos en este punto.

```

1  CORPUS = {
2      "DOC-1": {
3          "doc_type": "paper",
4          "text": '''
5          ONTOLOGIA DE LA ABSTRACCION Y SEMANTICA FORMAL
6          La programacion moderna se define como la gestion de capas de abstraccion para reducir la brecha ent
7          1. Paradigmas y Logica: La evolucion desde lo imperativo (como hacer) hacia lo declarativo (que obte
8          2. Gestion de la Complejidad: El software robusto se basa en la ortogonalidad, donde componentes sim
9          3. Concurrencia Moderna: Ante el limite fisico de los procesadores, la programacion ha pasado de la
10         En conclusion, programar es el arte de modelar intenciones mediante estructuras logicas que garantiz
11     },
12     "DOC-2": { ...
20     },
21     "DOC-3": { ...
52     },
53     "DOC-4": { ...
68     }
69 }
70
71 pd.DataFrame(
72     [
73         {"doc_id": k, "doc_type": v["doc_type"], "chars": len(v["text"]), "preview": v["text"][:100] + "..."}
74         for k, v in CORPUS.items()
75     ]
76 )

```

	doc_id	doc_type	chars	preview
0	DOC-1	paper	1264	\n ONTOLOGIA DE LA ABSTRACCION Y SEMANTICA FORMAL\n La programacion moderna se define co...
1	DOC-2	news	1183	\n CRISIS Y EVOLUCION: EL NUEVO PARADIGMA DE LA PROGRAMACION GLOBAL\n La industria del s...
2	DOC-3	manual	2379	\n MANUAL DE OPERACION TECNICA: PERSISTENCIA VECTORIAL EN SISTEMAS POSTGRESQL (NEON)\n 1...
3	DOC-4	faq	1451	\n PREGUNTAS FRECUENTES (FAQ): FUNDAMENTOS DE PROGRAMACION Y ABSTRACCION\n Pregunta: ¿Cu...

> 4. Cargar modelo de embeddings

Usaremos `SentenceTransformer("all-MiniLM-L6-v2")`, ampliamente utilizado para obtener embeddings ligeros y rápidos. Sentence Transformers documenta el uso de este modelo y muestra que sus embeddings tienen forma `[n, 384]`.

□cite□turn455150search10□turn455150search13□

↳ 1 celda oculta

> 5. Funciones auxiliares

Incluimos:

- conteo aproximado de tokens,
- generación de embeddings,
- limpieza opcional de tabla.

↳ 1 celda oculta

✓ 6. Estrategias de chunking

6.1 Fixed-size

`RecursiveCharacterTextSplitter` intenta respetar separadores jerárquicos antes de cortar. LangChain documenta este splitter para fragmentación basada en caracteres y separadores. □cite□turn455150search2□

```
1 def fixed_chunker(text, chunk_size=350, chunk_overlap=60):
2     splitter = RecursiveCharacterTextSplitter(
3         chunk_size=chunk_size,
4         chunk_overlap=chunk_overlap,
5         length_function=len,
```

```
6     separators=["\n\n", "\n", ". ", " ", ""]
7 )
8     return splitter.split_text(text)
```

> 6.2 Sentence-aware

Nunca corta una oración a la mitad.

Sirve muy bien para noticias, textos narrativos y documentos donde la oración es una unidad semántica fuerte.

↳ 1 celda oculta

> 6.3 Semantic

1. Divide en oraciones.
2. Calcula embedding por oración.
3. Calcula similitud coseno entre oraciones adyacentes.
4. Si la similitud cae por debajo de un umbral, corta.

Esta es la estrategia más costosa, pero suele producir bloques más coherentes en papers, manuales densos y textos con cambios temáticos internos.

↳ 1 celda oculta

> 7. Prueba rápida de las tres estrategias

Observa:

- número de chunks,
- tamaño promedio,
- y si los límites parecen razonables.

↳ 2 celdas ocultas

> 8. Funciones para insertar chunks en Neon

↳ 1 celda oculta

> 9. Pipeline experimental

La idea no es solo indexar, sino **comparar estrategias**.

Diseño del experimento sugerido

Cada estrategia se aplica al mismo corpus.

Luego evaluamos preguntas de prueba para ver:

- si recupera el documento correcto,
- si la similitud es razonable,
- si genera demasiados chunks,
- y si introduce redundancia.

Puedes empezar con estos parámetros y luego modificarlos:

- `fixed: chunk_size=350, chunk_overlap=60`
- `sentence: max_sent=3, overlap_sent=1`
- `semantic: threshold=0.55, min_sentences=2`

↳ 2 celdas ocultas

> 10. Indexar una estrategia

Primero prueba **una** sola estrategia.

Luego indexa las otras.

Pregunta: hagan una pausa aquí y piensen cuál estrategia ganará para `paper`, `news`, `manual` y `faq`.

↳ 3 celdas ocultas

✓ 11. Búsqueda semántica

Consultamos Neon usando:

- filtros SQL opcionales (`strategy`, `doc_id`, `doc_type`)
- ranking vectorial con distancia coseno.

Neon documenta `pgvector` para almacenar embeddings y ejecutar búsquedas vectoriales en PostgreSQL, y también muestra su uso para aplicaciones de búsqueda semántica. `□cite□turn455150search0□turn455150search14□`

```
1 def search_chunks(question, k=5, strategy=None, doc_id=None, doc_type=None):
2     query_emb = embed_text(question)
3
4     where = ["1=1"]
5     params = []
6
7     if strategy:
8         where.append("strategy = %s")
9         params.append(strategy)
10    if doc_id:
11        where.append("doc_id = %s")
12        params.append(doc_id)
13    if doc_type:
14        where.append("doc_type = %s")
15        params.append(doc_type)
16
17    sql = f'''
18    SELECT
19        doc_id,
20        doc_type,
21        strategy,
22        chunk_index,
23        token_count,
24        LEFT(content, 220) AS preview,
25        1 - (embedding <=> %s::vector) AS similarity
26    FROM chunks
27    WHERE {' AND '.join(where)}
28    ORDER BY embedding <=> %s::vector
29    LIMIT %s;
30    '''
31
32    full_params = [query_emb] + params + [query_emb, k]
33
34    with get_conn() as conn:
35        with conn.cursor() as cur:
36            cur.execute(sql, full_params)
37            rows = cur.fetchall()
38
39    return pd.DataFrame(rows, columns=[
```

```
10      "doc_id" "doc_type" "strategy" "chunk_index" "token_count" "previous" "similarity"

1  queries = [
2      "¿Qué establece el Isomorfismo de Curry-Howard respecto a los sistemas de tipos estáticos?",
3      "¿Qué porcentaje de nuevas vulnerabilidades en empresas como Microsoft y Google se atribuyen actualmente",
4      "¿Qué índice se debe implementar en PostgreSQL para optimizar la búsqueda aproximada (ANN) en conjuntos",
5      "¿Cuál es la principal ventaja de utilizar el modelo de Actores o CSP frente al uso de hilos tradicional",
6  ]
7
8  for q in queries:
9      print("\n" + "="*90)
10     print("PREGUNTA:", q)
11     display(search_chunks(q, k=5))
```


=====

PREGUNTA: ¿Qué establece el Isomorfismo de Curry-Howard respecto a los sistemas de tipos estáticos?

	doc_id	doc_type	strategy	chunk_index	token_count	preview	similarity
0	DOC-1	paper	semantic	1	24	Bajo el Isomorfismo de Curry-Howard, los sistemas de tipos estaticos no son solo restricciones, sino pruebas matemat...	0.871861
1	DOC-4	faq	sentence	4	34	Pregunta: ¿Que establece el Isomorfismo de Curry-Howard? Respuesta: Establece una relacion directa entre la logica m...	0.743649
2	DOC-1	paper	sentence	1	43	Paradigmas y Logica: La evolucion desde lo imperativo (como hacer) hacia lo declarativo (que obtener) busca la purez...	0.732371
3	DOC-1	paper	fixed	1	43	1. Paradigmas y Logica: La evolucion desde lo imperativo (como hacer) hacia lo declarativo (que obtener) busca la pu...	0.725187
4	DOC-4	faq	sentence	3	48	Pregunta: ¿Cual es la ventaja de usar el modelo de Actores o CSP en lugar de hilos (threads) tradicionales? Respuest...	0.690995

=====

PREGUNTA: ¿Qué porcentaje de nuevas vulnerabilidades en empresas como Microsoft y Google se atribuyen actualmente a la gestión de memoria?

✓ 12. Evaluación comparativa de estrategias

	doc_id	doc_type	strategy	chunk_index	token_count	preview	similarity
0	DOC-2	news	fixed	2	27	. Gigantes como Microsoft y Google ya reportan que mas del 70% de sus nuevas vulnerabilidades provienen de gestion d...	0.766793
1	DOC-2	news	sentence	0	78	CRISIS Y EVOLUCION: EL NUEVO PARADIGMA DE LA PROGRAMACION GLOBAL\nLa industria del software atraviesa su tra...	0.763424
2	DOC-2	news	semantic	0	78	CRISIS Y EVOLUCION: EL NUEVO PARADIGMA DE LA PROGRAMACION GLOBAL\nLa industria del software atraviesa su tra...	0.763424

```

1 EVAL_SET = [
2     {"question": "¿Que busca la evolucion hacia lo declarativo?", "expected_doc": "DOC-1"},
3     {"question": "¿Que lenguajes estan cayendo en desuso por su gestion de memoria?", "expected_doc": "DOC-2"},
4     {"question": "¿Que operador se utiliza para determinar la distancia coseno?", "expected_doc": "DOC-3"},
5     {"question": "¿Por que es importante la ortogonalidad en el software?", "expected_doc": "DOC-4"}
6 ]
7
8 pd.DataFrame(EVAL_SET)

```

0	DOC-3	manual	fixed	0	16	MANUAL DE OPERACION TECNICA: PERSISTENCIA VECTORIAL EN SISTEMAS POSTGRESQL (NEON)\n1. CONFIGURACION DEL ENTO...	0.684753
0						¿Que busca la evolucion hacia lo declarativo? DOC-1	0.662795
1						¿Que lenguajes estan cayendo en desuso por su gestion de memoria? DOC-2	0.662795
2						¿Que operador se utiliza para determinar la distancia coseno? DOC-3	0.662795
3	DOC-3	manual	fixed	1	39	¿Por que es importante la ortogonalidad en el software? 1. CONFIGURACION DEL ENTORNO DE ALMACENAMIENTO\nLa infraestructura requiere una instancia de PostgreSQL serv...	0.579320

```
1 def evaluate_strategy(strategy_name, k=3):
2     rows = []
3
4     for item in EVAL_SET:
5         result = search_chunks(item["question"], k=k, strategy=strategy_name)
6         if result.empty:
7             rows.append({
8                 "strategy": strategy_name,
9                 "question": item["question"],
10                "expected_doc": item["expected_doc"],
11                "top1_doc": None,
12                "hit_at_k": 0,
13                "mrr": 0.0,
14                "top1_similarity": None
15            })
16            continue
17
18        retrieved_docs = result["doc_id"].tolist()
19        top1_doc = retrieved_docs[0]
20        hit = int(item["expected_doc"] in retrieved_docs)
21
22        rr = 0.0
23        for rank, doc in enumerate(retrieved_docs, start=1):
24            if doc == item["expected_doc"]:
25                rr = 1 / rank
26                break
27
28        rows.append({
29            "strategy": strategy_name,
30            "question": item["question"],
31            "expected_doc": item["expected_doc"],
32            "top1_doc": top1_doc,
33            "hit_at_k": hit,
34            "mrr": rr,
35            "top1_similarity": float(result.iloc[0]["similarity"])
36        })
37
38    detail = pd.DataFrame(rows)
39
40    # métricas estructurales
```

```
41     with get_conn() as conn:
42         structural = pd.read_sql(
43             '''
44             SELECT
45                 strategy,
46                 COUNT(*) AS total_chunks,
47                 AVG(token_count)::float AS avg_tokens,
48                 STDDEV(token_count)::float AS std_tokens
49             FROM chunks
50             WHERE strategy = %s
51             GROUP BY strategy;
52             ''',
53             conn,
54             params=[strategy_name]
55         )
56
57     summary = detail.groupby("strategy").agg(
58         hit_at_k=("hit_at_k", "mean"),
59         mrr=("mrr", "mean"),
60         mean_top1_similarity=("top1_similarity", "mean")
61     ).reset_index()
62
63     if not structural.empty:
64         summary = summary.merge(structural, on="strategy", how="left")
65
66     return detail, summary
```

```
1 all_summaries = []
2 all_details = []
3
4 for strat in ["fixed", "sentence", "semantic"]:
5     detail, summary = evaluate_strategy(strat, k=3)
6     all_details.append(detail)
7     all_summaries.append(summary)
8
9 eval_detail_df = pd.concat(all_details, ignore_index=True)
10 eval_summary_df = pd.concat(all_summaries, ignore_index=True).sort_values("mrr", ascending=False)
11
12 display(eval_summary_df)
```

	strategy	hit_at_k	mrr	mean_top1_similarity	total_chunks	avg_tokens	std_tokens
1	sentence	0.75	0.625	0.623623	25	48.880000	15.103311
2	semantic	0.75	0.625	0.676245	23	37.304348	19.907593
0	fixed	0.50	0.250	0.604262	26	34.076923	9.363431

Próximos pasos:

[Generar código con eval_summary_df](#)[New interactive sheet](#)

1 eval_detail_df

	strategy	question	expected_doc	top1_doc	hit_at_k	mrr	top1_similarity
0	fixed	¿Que busca la evolucion hacia lo declarativo?	DOC-1	DOC-4	1	0.5	0.550758
1	fixed	¿Que lenguajes estan cayendo en desuso por su gestion de memoria?	DOC-2	DOC-4	0	0.0	0.573188
2	fixed	¿Que operador se utiliza para determinar la distancia coseno?	DOC-3	DOC-4	0	0.0	0.570435
3	fixed	¿Por que es importante la ortogonalidad en el software?	DOC-4	DOC-1	1	0.5	0.722665
4	sentence	¿Que busca la evolucion hacia lo declarativo?	DOC-1	DOC-4	1	0.5	0.508597
5	sentence	¿Que lenguajes estan cayendo en desuso por su gestion de memoria?	DOC-2	DOC-1	0	0.0	0.665928
6	sentence	¿Que operador se utiliza para determinar la distancia coseno?	DOC-3	DOC-3	1	1.0	0.569676
7	sentence	¿Por que es importante la ortogonalidad en el software?	DOC-4	DOC-4	1	1.0	0.750290
8	semantic	¿Que busca la evolucion hacia lo declarativo?	DOC-1	DOC-3	1	0.5	0.601899
9	semantic	¿Que lenguajes estan cayendo en desuso por su gestion de memoria?	DOC-2	DOC-4	0	0.0	0.573188
10	semantic	¿Que operador se utiliza para determinar la distancia coseno?	DOC-3	DOC-3	1	1.0	0.779603
11	semantic	¿Por que es importante la ortogonalidad en el software?	DOC-4	DOC-4	1	1.0	0.750290

Próximos pasos:

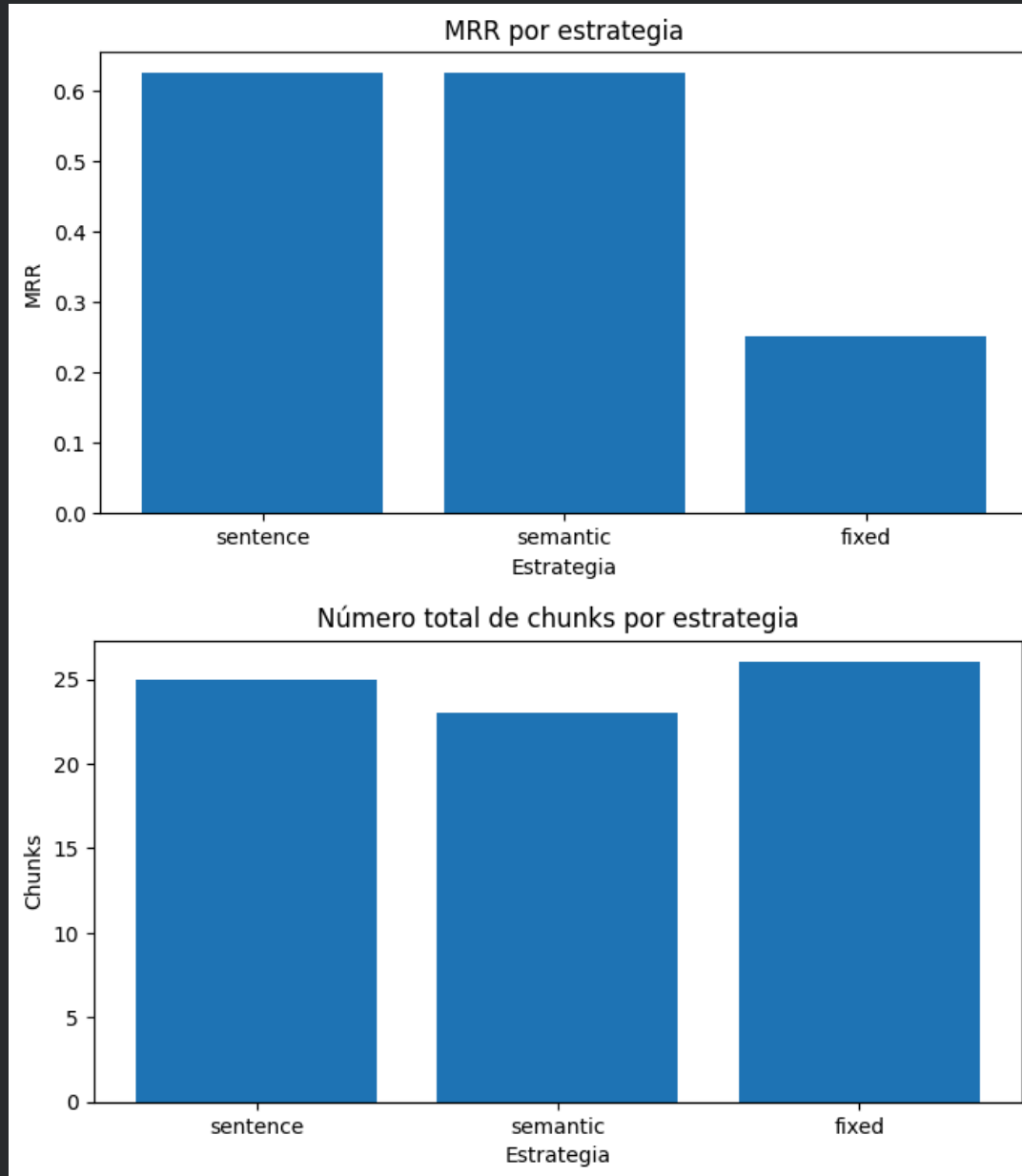
[Generar código con eval_detail_df](#)[New interactive sheet](#)

13. Visualización rápida de resultados

Aquí puedes discutir en clase:

- cuál tuvo mayor **MRR**,
- cuál produjo más chunks,
- cuál generó chunks más grandes,
- y si la mejora vale el costo.

```
1 plot_df = eval_summary_df.copy()
2
3 plt.figure(figsize=(8,4))
4 plt.bar(plot_df["strategy"], plot_df["mrr"])
5 plt.title("MRR por estrategia")
6 plt.xlabel("Estrategia")
7 plt.ylabel("MRR")
8 plt.show()
9
10 plt.figure(figsize=(8,4))
11 plt.bar(plot_df["strategy"], plot_df["total_chunks"])
12 plt.title("Número total de chunks por estrategia")
13 plt.xlabel("Estrategia")
14 plt.ylabel("Chunks")
15 plt.show()
```



✓ 14. Diagnóstico de semantic chunking

Prueba con distintos umbrales para ver cómo cambian los quiebres temáticos.

```
1 thresholds = [0.40, 0.55, 0.70]
2 diag_results = []
3
4 for th in thresholds:
5     chunks = semantic_chunker(CORPUS["DOC-1"]["text"], threshold=th, min_sentences=2)
6     diag_results.append({
7         "threshold": th,
8         "n_chunks": len(chunks),
9         "avg_tokens": int(np.mean([approx_token_count(c) for c in chunks]))
10    })
11
12 pd.DataFrame(diag_results)
```

	threshold	n_chunks	avg_tokens
0	0.40	4	45
1	0.55	5	36
2	0.70	5	36

```
1 semantic_diagnostics(CORPUS["DOC-1"]["text"], threshold=0.55)
```

i		sent_a	sent_b	similarity	break
0	0	ONTOLOGIA DE LA ABSTRACCION Y SEMANTICA FORMAL\n La programacion moderna s	Este proceso se sustenta en tres pilares fundamentales:\n 1.	0.304473	True
1	1	Este proceso se sustenta en tres pilares fundamentales:\n 1.	Paradigmas y Logica: La evolucion desde lo imperativo (como hacer) hacia lo decl	1.000000	False
2	2	Paradigmas y Logica: La evolucion desde lo imperativo (como hacer) hacia lo decl	Bajo el Isomorfismo de Curry-Howard, los sistemas de tipos estaticos no son solo	0.498796	True
3	3	Bajo el Isomorfismo de Curry-Howard, los sistemas de tipos estaticos no son solo	2.	0.374735	True
4	4	2.	Gestion de la Complejidad: El software robusto se basa en la ortogonalidad, dond	0.115506	True
5	5	Gestion de la Complejidad: El software robusto se basa en la ortogonalidad, dond	Lenguajes como Rust o Haskell ejemplifican esto al tratar la seguridad de memori	0.304257	True
6	6	Lenguajes como Rust o Haskell ejemplifican esto al tratar la seguridad de memori	3.	0.338992	True
7	7	3.	Concurrencia Moderna: Ante el limite fisico de los procesadores, la programacion	0.076297	True
8	8	Concurrencia Moderna: Ante el limite fisico de los procesadores, la programacion	En conclusion, programar es el arte de modelar intenciones mediante estructuras	0.491245	True

✓ 15. Comparación NLTK vs spaCy para sentence-aware

El taller original propone comparar detectores de oraciones. Aquí lo hacemos en el mismo notebook.

```

1 nltk_chunks = sentence_chunker(CORPUS["DOC-2"]["text"], max_sent=3, overlap_sent=1)
2 spacy_chunks = sentence_chunker_spacy(CORPUS["DOC-2"]["text"], max_sent=3, overlap_sent=1)
3
4 print("NLTK chunks:", len(nltk_chunks))
5 print("spaCy chunks:", len(spacy_chunks))
6
7 print("\n--- Primer chunk NLTK ---\n")
8 print(nltk_chunks[0])
9
10 print("\n--- Primer chunk spaCy ---\n")
11 print(spacy_chunks[0])

```

NLTK chunks: 4
spaCy chunks: 4

```
--- Primer chunk NLTK ---
```

```
CRISIS Y EVOLUCION: EL NUEVO PARADIGMA DE LA PROGRAMACION GLOBAL
```

```
La industria del software atraviesa su transformacion mas disruptiva en decadas. Agencias de ciberseguridad internacionales han emitido di
```

```
--- Primer chunk spaCy ---
```

```
CRISIS Y EVOLUCION: EL NUEVO PARADIGMA DE LA PROGRAMACION GLOBAL
```

```
La industria del software atraviesa su transformacion mas disruptiva en decadas. Agencias de ciberseguridad internacionales han emitido di
```

✓ 16. Experimentos sugeridos para realizar

Experimento A

Compara `fixed-size`:

- `chunk_size=256, overlap=32`
- `chunk_size=350, overlap=60`
- `chunk_size=512, overlap=64`

Experimento B

Compara `sentence-aware`:

- `max_sent=2`
- `max_sent=3`
- `max_sent=5`

Experimento C

Compara `semantic`:

- `threshold=0.40`
- `threshold=0.55`
- `threshold=0.70`

En cada caso registren:

- número de chunks,
- promedio de tokens,
- hit@3,
- MRR,
- observación cualitativa sobre coherencia.

```
1 def run_fixed_experiments(configs):
2     results = []
3     for cfg in configs:
4         clear_chunks()
5         index_strategy("fixed", CORPUS, params=cfg, reset=False)
6         _, summary = evaluate_strategy("fixed", k=3)
7         row = summary.iloc[0].to_dict()
8         row.update(cfg)
9         results.append(row)
10    return pd.DataFrame(results).sort_values("mrr", ascending=False)
11
12    fixed_experiments = [
13        {"chunk_size": 256, "chunk_overlap": 32},
14        {"chunk_size": 350, "chunk_overlap": 60},
15        {"chunk_size": 512, "chunk_overlap": 64},
16    ]
17
18    run_fixed_experiments(fixed_experiments)
```

Tabla chunks vaciada.

Insertados 8 chunks -> DOC-1 [fixed]

Insertados 8 chunks -> DOC-2 [fixed]

Insertados 12 chunks -> DOC-3 [fixed]

Insertados 8 chunks -> DOC-4 [fixed]

Tabla chunks vaciada.

Insertados 5 chunks -> DOC-1 [fixed]

Insertados 6 chunks -> DOC-2 [fixed]

Insertados 9 chunks -> DOC-3 [fixed]

Insertados 6 chunks -> DOC-4 [fixed]